

SCHEME AND EXCEPTIONS Meta

COMPUTER SCIENCE MENTORS 61A

November 17 – November 22, 2025

Example Timeline

- Ice Breaker [3 min]

If you want, start off with an icebreaker of your choice!

Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!

- Mini-lecture: Interpreters [10 min]
- Q1: Link [6 min]
- Q2: Scheme Eval/Apply [4 min]
- Mini-Lecture: Macros
- Q3: combine-num [7 min]
- Q4:
- Q5:

1 Scheme Interpreters

1. Based on the Link class that is below, write the scheme expression represented by the Link class. Assume that this is the Link class in Python that we will be using:

```
class Link:
    '''A Scheme list is a Link in which rest is a Link or nil.'''
    empty = ()
    def __init__(self, first, rest=empty):
        self.first = first
        self.rest = rest

    # There are also __str__, __repr__, and map methods, omitted here.

nil = Link.empty
>>> Link('*', Link(5, Link(Link('-', Link(10, Link(2, nil))), nil)))

(* 5 (- 10 2))

>>> Link('or', Link(Link('>', Link(5, Link(2, nil))), Link(Link('/',
    Link(1, Link(2, nil))), nil)))

(or (> 5 2) (/ 1 2))

>>> Link('+', Link(1, Link(Link('*', Link(Link('-', Link(5, Link(2,
    nil))), Link(Link('+', Link(3, Link(1, nil))), nil))), Link(5,
    nil))))

(+ 1 (* (- 5 2) (+ 3 1)) 5)
```

Teaching Tips

- Explain the Link class thoroughly and if you have time, explain what its connection is to interpreters and the scheme project.

Suggested Time: 6 min

2. Circle the number of calls to `scheme_eval` and `scheme_apply` for the code below.

```
(+ 1 2)

scheme_eval    1  3  4  6
scheme_apply   1  2  3  4

4 scheme_eval, 1 scheme_apply.
```

Suggested Time: 5 min

1. Implement the macro function `combine-num`, which takes in an unquoted list of positive integers and returns a number whose digits are the items of the list in reverse order.

```
; Doctests
scm> (combine-num (1 2))
; 21
scm> (combine-num (2 5 3 5))
; 5352
scm> (combine-num (1))
; 1
scm> (+ (combine-num (1 2 3 4)) 5)
; 4326      # (4321 + 5)
```

```
(define-macro (combine-num lst)
```

```
)
```

```
(define-macro (combine-num lst)
  (if (null? lst) 0
      `(+ ,(car lst) (* 10 (combine-num ,(cdr lst)))))
  )
)
```

Suggested Time: 7 min